FIND-R: Failure Identification and Directed Repair via Representation Probing in Neural Networks

Hakka Madan
Computer Science Department (SCAI)
Arizona State University
United States
hmadan2@asu.edu

Kunal Garg
Aerospace Department (SEMTE)
Arizona State University
United States
kgarg24@asu.edu

Abstract: Neural networks have expanded the use of autonomous systems in complex real-world settings, yet provable guarantees for such systems remain difficult to obtain because verifying the underlying networks remains intractable, motivating probing as a way to examine the recoverability of latent properties from internal representations. Probing has been well studied in vision and language processing, but rarely as a diagnostic tool in robotics. This work presents a probing-inspired, structured diagnosis-and-repair framework for safety-critical robotics systems, built on the central model-agnostic hypothesis that a property directly linked to the training objective should remain consistently recoverable throughout the network without abrupt degradation across layers. We investigate the hypothesis through case studies of two neural training architectures, GCBF+ and InforMARL, proposed for safe control design for multi-robot systems using graph neural networks. We illustrate that probing can localize internal failures of a network to capture safety, characterize false modes, inform targeted repairs that reduce failures by up to 95% for GCBF+ and 65% for InforMARL, and run up to 90% faster than existing verification tools for GCBF+ and 80% for InforMARL.

Keywords: Formal verification, Graph neural networks, Neural network probing

1 Introduction

Autonomous multi-agent systems (MAS) are increasingly being used to solve large tasks through parallel and distributed decision-making [1, 2]. In high-throughput and low-latency applications, the control inputs for MAS are often generated by a neural network (NN)-based controller [3, 4], making reliability and interpretability of the NN necessary for its deployment in safety-critical applications. Strong empirical task performance of learned controllers for applications such as reachability while avoidance (RWA) as demonstrated in [5, 4, 6] does not necessarily imply correctness of the latent safety structure required for trustworthy decision-making [7]. Misalignment between performance and internal representations of learned controllers can limit their use in out-of-distribution environments [8]. While full formal verification can, in principle, answer this question, it is extremely slow and intractable across complex deep NN architectures with nonlinear activations [9, 10].

Probing has emerged as a standard post hoc methodology for accessing the layer-wise representations of a NN and quantifying the recoverability of latent properties, the factors that determine how the output is generated in a NN [11, 12]. Probing involves training lightweight auxiliary NNs, termed probes, on intermediate representations of the NN being probed, where the probe architecture depends on the complexity and nature of the property being recovered and the probe's training data [13, 14]. Despite probing being a well-established interpretability framework in language and vision [11, 15, 16], it remains largely confined to non-robotics applications. Through this work, we aim to answer the following questions: (1) whether strong empirical performance in learned MAS controllers can coexist with incomplete or misaligned encoding of a formal safety structure. (2) whether false modes can be collected and tied to this coexistence.

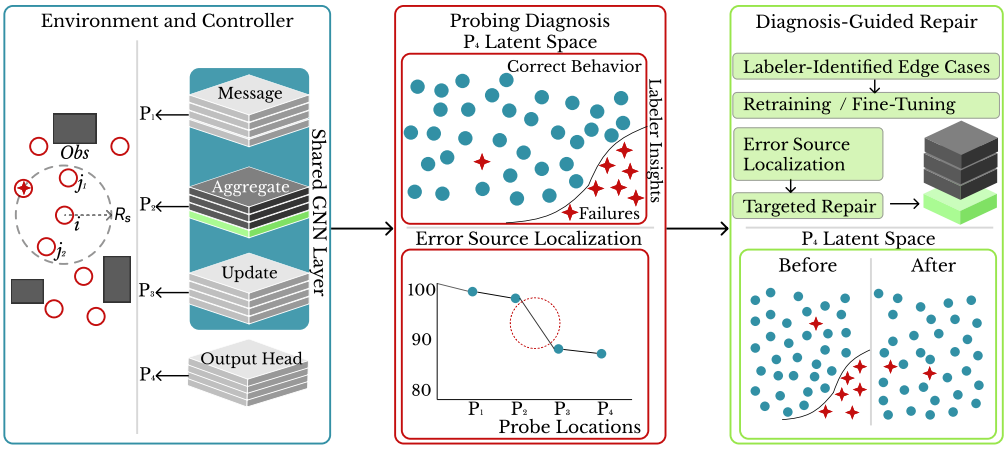


Figure 1: Probing-based diagnosis framework for graph neural networks (GNNs): rollout samples are labeled as safe or unsafe, failure samples are collected, and false modes are analyzed, the indicated GNN layers are probed to localize the source of misalignment between the intended and the learned property, and the underlying activations are examined to inform targeted repairs.

Prior probing methods have remained largely descriptive, focusing primarily on the presence, absence, or the degree of recoverability of latent properties [17, 16]. To the best of our knowledge, existing probing methods treat the intermediate representation labels as supervision targets for the probe [11, 18], rather than examining how the labeling process itself might reveal clusters of edge cases or other actionable insights. On the other hand, complete formal verification is the benchmark for checking the correctness of NN approximated functions over the state space [19], but is constrained by the size, complexity, and the variety of non-linearities of activations in modern NNs [20, 21]. Instead, an important middle-ground solution can be developed that leverages probing's interpretability, enables targeted repairs, and remains tractable, unlike full formal verification.

This work addresses the identified gap by extending probing into a sequential diagnosis-and-repair framework for safety-critical learned controllers. The central model-agnostic hypothesis of this work is that the recoverability of a latent property tied to the training objective of a NN should not undergo severe abrupt degradation across network layers [17]; such sharp plunges can signal encoding bottlenecks, insufficient supervision, or a mismatch between the learned and the intended property. Consequently, probing is used to diagnose where the safety deteriorates, and to guide targeted repairs through analysis of probe behavior and labeler-driven insights. Here, repair refers to reducing the NN's false modes by improving its internal representations of the latent property. This formulation is examined through case studies on GCBF+ [6] and InforMARL [5], where diagnostic outcomes suggest the internal sources of failure. The proposed work assumes that lightweight probes induce negligible compounding bias and do not overfit [22]. The contributions of this work are threefold: (1) A probing-based diagnosis framework for safety-critical MAS with learned controllers that evaluates whether safety remains consistently recoverable across all internal layers, localizes abrupt plunges, and characterizes false modes. (2) A sequentially structured repair framework that translates the diagnosis into an empirically sound NN repair. (3) A tractability and computation time comparison of the proposed framework against formal verification and feasibility analysis, demonstrating more scalability and speed.

2 Related Work

Foundational probes introduced lightweight linear classifiers on intermediate NN activations to measure the recoverability of target properties across layers, framing probing as a possible diagnostic tool [23]. Subsequent works extend this view to domains such as program semantics and neural combinatorial models [12, 18]. Probe-design studies emphasize that probing conclusions depend

on probe capacity, control tasks, loss design, and evaluation protocol; high accuracy indicates that the information is retrievable but not how it is used, and should be interpreted relative to latent variables in the generative process [24, 11, 14, 25]. **Beyond conventional probing**, [26, 15, 27, 28, 29] analyze internal topologies, decision boundaries, attention-head specialization, intervention-guided localization, and latent hierarchial structure, but remain largely centered on language models. **Graph-oriented probing** studies node-wise, path-wise, and structural properties in learned graph representations, yet remains focused on representational analysis [17, 30]. **Probing in robotics** is limited to a small body of work in reinforcement learning (RL) that does not meaningfully address probe design, or safety-critical diagnosis [31]. Consequently, prior work leaves a gap for a structured probing-based diagnosis and intervention framework, particularly for safety-critical MAS.

Behavioral decomposition methods add interpretability to policies by mapping disentangled activation paths to individual behaviors [32]. Feature-attribution methods decompose rewards or assign salience scores to inputs influencing robot's decisions, including graph based policies [33, 34], while neuro-symbolic methods generate temporal logic or symbolic explanations for robot's behaviors [35, 36]. Emerging vision-language and large-language model approaches explain robot decisions through detailed reasoning traces [37], and causal explanation methods relate robot behavior to human understandable objects, actions, or task conditions [38]. Collectively, these works explain why or how a decision was made rather than whether the correct task-relevant property was encoded to a sufficient extent to produce it, and generally do not provide diagnosis or repair frameworks.

Satisfiability Modulo Theories (SMT)-based full verification remains limited in expressiveness and tractability for neural controllers [39]. Although NN verification advanced, scaling beyond piecewise-linear activations such as ReLU remains difficult, particularly for closed-loop temporal verification of reactive Deep RL or Deep NN policies that require policy unrolling across timesteps [21, 40]. Recent synthesis and verification frameworks [41, 7] improve efficiency but still rely on ReLU activations, activation region enumeration, and limit synthesis to less expressive nonlinearities. Verification is even more challenging for GNNs, as most methods assume fixed graph structures and linear constraints, while bounded verification of nonlinear GNNs with dynamic topology remains NP-complete and highly intractable [9, 42, 43, 44]. On the other hand, learning-based feasibility constraint formulations improve feasibility under input bounds, but do not yield scalable end-to-end formal verification of NNs [45, 46, 47].

3 Methodology

We hypothesize that a latent property aligned with the objective of the NN should remain consistently recoverable across its layer-wise representations without abrupt degradation across depth. We propose probing as a classification problem with safety (in terms of collision avoidance, defined with binary labels in (2)) as the latent property being recovered for GCBF+ [6] and InforMARL [5], two distributed neural training architectures, that have been proposed for safe control design for MAS. Both architectures share the same underlying GNN backbone [6] (see Figure 1), but differ in the framework-specific head, GNN layer dimensions, and training mechanisms. We describe the broadly similar probing methodology used for the two case studies and defer framework-specific architectural details to the results section. The shared GNN layer in the two methods consists of three sequential multi-layer perceptron (MLP) layers, commonly referred to as Message, Aggregate, and Update. For both case studies, we probe these layers and the framework-specific output heads, namely the CBF head in GCBF+ and the Critic head in InforMARL.

MAS environment setup: Let $N \in \mathbb{N}$ denote the total number of agents and $\mathcal{V} = \{1, \dots, N\}$ the corresponding agent index set. Further, let $\mathcal{T} = \{0, \dots, T\}$ denote the discrete inference horizon, where $T \in \mathbb{N}$ is the maximum timestep. Each agent $i \in \mathcal{V}$ senses only local entities within a sensing radius $R_i \in \mathbb{R}_{>0}$, which induces a local interaction graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$, where $\mathcal{V}_t$ denotes the set of locally sensed entities at time $t \in \mathcal{T}$, henceforth referred to as neighbor j, and $\mathcal{E}_t \subseteq \mathcal{V}_t \times \mathcal{V}_t$ denotes the associated interaction edges. Let $p_i(t), v_i(t) \in \mathbb{R}^2$ denote, respectively, the position and velocity of agent i at time t, and let $p_j(t), v_j(t) \in \mathbb{R}^2$ denote those of neighbor j. For each

edge $(i, j) \in \mathcal{E}_t$, the relative position and relative velocity is defined as $p_{ij}(t) := p_i(t) - p_j(t)$ and $v_{ij}(t) := v_i(t) - v_j(t)$, and the corresponding edge feature as $e_{ij}(t) = [\, p_{ij}(t),\, v_{ij}(t)\,] \in \mathbb{R}^4$.

Problem formulation: We formulate probing as a binary classification problem in which the target is the safety label of agent i at time t. Let $z \in \mathcal{Z} \subseteq \mathbb{R}^d$ denote the input to the probe, where $\mathcal{Z}$ is set of the GNN's layer representations and $d > 0$ is the dimensionality of the layer being probed. The probe is then defined as a function $b : \mathcal{Z} \to \{0, 1\}$ that maps the GNN representations to the binary target variable $y \in \{0, 1\}$. The probe output is denoted as $\hat{y} = b(z)$.

Data collection: To define the target label, we construct a binary safety variable $y_i(t) \in \{0, 1\}$ from pairwise agent-agent (AA) and agent-obstacle (AO) interactions involving the agent i. Let $r_{ij}(t) \in \mathbb{R}$ and $D_{ij}(t) \in \mathbb{R}_{\geq 0}$, respectively, denote the radial component of relative velocity, the component that dictates collisions, and the associated minimum stopping distance where $\alpha_i, \alpha_j \in \mathbb{R}_{>0}$ denote the corresponding maximum acceleration bounds, defined in [48] as:

$$r_{ij}(t) = \frac{p_{ij}(t)^\top v_{ij}(t)}{\|p_{ij}(t)\|}, \qquad D_{ij}(t) = \frac{r_{ij}(t)^2}{2\,(\alpha_i + \alpha_j)}. \tag{1}$$

Here, $r_{ij}(t) > 0$ indicates a diverging pair and $r_{ij}(t) < 0$ indicates an approaching pair. Using these pairwise quantities, inspired by [48], we define binary pairwise safety indicators $S_{ij}^{\mathrm{AA}}(t) \in \{0, 1\}$ and $S_{ij}^{\mathrm{AO}}(t) \in \{0, 1\}$, for AA and AO interactions, respectively, where, $S_{ij}(t) = 1$ indicates a safe interaction between i and j when $\|p_{ij}(t)\| - D_{ij}(t) \geq D_s$ and $S_{ij}(t) = 0$ otherwise, where $D_s \in \mathbb{R}_{\geq 0}$ is the entity (i.e., another moving agent or stationary obstacles) specific safety threshold defined in [6]. Let $\mathcal{N}_i^{\mathrm{AA}}(t)$ and $\mathcal{N}_i^{\mathrm{AO}}(t)$ denote the sets of agent and obstacle neighbors of agent i within R_i at time t, respectively. The binary safety label assigned to the each agent i at time t is then given by:

$$y_i(t) = \mathbb{I}\left[\bigwedge_{j \in \mathcal{N}_i^{\mathrm{AA}}(t)} S_{ij}^{\mathrm{AA}}(t) \;\wedge \bigwedge_{j \in \mathcal{N}_i^{\mathrm{AO}}(t)} S_{ij}^{\mathrm{AO}}(t) \right], \tag{2}$$

where $\mathbb{I}[\cdot]$ denotes the indicator function. Thus, $y_i(t) = 1$ only when all locally relevant AA and AO interactions involving agent i are safe, and $y_i(t) = 0$ otherwise. Let $\mathcal{Q} \subseteq \mathcal{V} \times \mathcal{T}$ denote the set of labeler-identified unsafe samples, defined as:

$$\mathcal{Q} := \{(i, t) \in \mathcal{V} \times \mathcal{T} : y_i(t) = 0\}. \tag{3}$$

Probe design, training, and evaluation: To design an optimal probe architecture for the application at hand, we first probe the penultimate layers of the CBF head of GCBF+ and the Critic head of InforMARL, which serve as the final latent representations before the respective framework outputs. Linear probes offer maximum simplicity and interpretability but are insufficient for recovering nonlinear latent properties from intermediate representations [14, 25]. Nonlinear probes such as MLPs, GNNs, and transformer-based probes provide greater expressiveness but come at the cost of increased complexity [14]. To this end, we consider a linear and a nonlinear MLP probe, and select the final probe architecture based on their predictive performance under a common training setup. We evaluate MLP probes with varying depths, widths, and compression schedules, and report the used architecture in Section 4. All considered probe architectures (more details in Appendix A.1) are trained on the same layer-wise encodings extracted from our implementations of GCBF+ and InforMARL using [6], which we henceforth refer to as **pre-trained** GNNs. We train the probes with binary cross-entropy loss, LeakyReLU activations in the hidden layers, and a sigmoid activation at the output layer. The probe's performance is evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-score** defined in [49]. For probing the safety property, **Recall** serves as the probe's primary evaluation criterion, since classifying unsafe states as safe is catastrophic, as it leads to possible collisions, while classifying safe states as unsafe leads to a conservative yet safe behavior. We apply the same probe to the penultimate layers of all four MLP blocks in both case studies to examine how safety recoverability evolves across the network.

Formal verification setup: Let $\bar{x} \in \mathcal{X}^N$ denote the concatenated joint state of the N-agent system, and let $\bar{x}_{\mathcal{N}_i} \in \mathcal{X}^M$ denote the local joint state of agent i and its neighborhood $\mathcal{N}_i$, following the GCBF+ formulation, where $\mathcal{N}_i$ is defined as the set of M closest neighbors to i. Let

$u_i \in \mathcal{U}_i \subset \mathbb{R}^m$ denote the admissible control input for agent i. The function $h_i : \mathcal{X}^M \to \mathbb{R}$ denotes the `graph` CBF (GCBF) for agent i, where the safe set is defined as 0-superlevel set $\mathcal{C} := \left\{ \bar{x} \in \mathcal{X}^N \,\middle|\, h_i(\bar{x}_{\mathcal{N}_i}) \geq 0,\ \forall i \in \mathcal{V} \right\}$. For an extended class-$\mathcal{K}$ function $\alpha : \mathbb{R} \to \mathbb{R}$, and control-affine dynamics $\dot{x}_i = f(x_i) + g(x_i)u_i$, the primary GCBF condition for safety is

$$L_f h_i(\bar{x}_{\mathcal{N}_i}) + L_g h_i(\bar{x}_{\mathcal{N}_i})\ u_i + \alpha(h_i(\bar{x}_{\mathcal{N}_i})) \geq 0, \tag{4}$$

where $L_f h_i$ and $L_g h_i$ denote the Lie derivatives of h_i along f and g evaluated at $\bar{x}_{\mathcal{N}_i}$ [6].

Unlike GCBF+, InforMARL does not explicitly learn a CBF, making a CBF-like condition in (4) not well-defined for the learned critic, since the critic is optimized for long-horizon return estimation rather than forward invariance of $\mathcal{C}$. Let $\pi_i : \mathcal{X}^N \to \mathcal{U}_i$ denote the actor policy for agent i, and let $V_i : \mathcal{X}^N \to \mathbb{R}$ denote the corresponding critic value. To enable a structurally comparable analysis across the two architectures, we instead define a critic-based surrogate $\tilde{V}_i : \mathcal{X}^N \to \mathbb{R}$ by centering V_i along the trajectory as $\tilde{V}_i(\bar{x}; x(0)) = V_i(\bar{x}) - \tau_i(x(0))$, where τ_i denotes the trajectory-specific reference point for agent i, defined as $\tau_i(\bar{x}(0)) = \max_{t\in[0,T]} V_i(\bar{x}(t); \bar{x}(0))$. Higher values of $V_i(\bar{x})$ correspond to larger expected return under π_i. Consequently, $\tilde{V}_i(\bar{x}) > 0$ indicates that the critic value at $\bar{x}$ is above its trajectory-level reference point. Note that $\tilde{V}_i$ does not provide formal closed-loop safety guarantees. Rather, it serves as a surrogate quantity for InforMARL that can be analyzed analogously to the formally learned safety certificate in GCBF+. We denote by $\mathcal{D} \subseteq \mathcal{X}^N$ the verifier domain, consisting of the joint states visited along rollout. We then define the following CBF-inspired surrogate condition for each agent:

$$L_f \tilde{V}_i(\bar{x}) + L_g \tilde{V}_i(\bar{x})\, u_i + \alpha\Big(\tilde{V}_i(\bar{x})\Big) \geq 0. \tag{5}$$

Full formal verification: Let h_i^θ denote the learned GCBF for agent i, and let $\pi_i^{\theta'}$ denote the learned GCBF+ policy. Inspired by [50, 51, 52], to verify the GNN for learned GCBF, we instantiate the closed-loop control $u_i = \pi_i^{\theta'}(\bar{x})$ and over all timesteps t in the trajectory, use the baseline SMT solvers to check whether the CBF condition in (4) holds for all joint states in the safe set for each agent. For InforMARL, we verify the critic-based surrogate condition analogously. Let V_i^ϕ and $\tilde{V}_i^\phi(\bar{x})$, respectively, denote the learned per-agent critic and its trajectory-centered version, and $\pi_i^{\phi'}$ denote the learned InforMARL policy. We instantiate the closed-loop control $u_i = \pi_i^{\phi'}(\bar{x})$ and check whether the surrogate condition (5) holds over the verifier domain for each agent.

Point-wise feasibility: For GCBF+, point-wise feasibility does not certify correctness of the learned CBF over set $\mathcal{C}$, but certifies that for a given $\bar{x}$, there exists at least one admissible control input satisfying the CBF condition. Formally, for each agent $i \in V$, for each t, we take the corresponding $\bar{x}$ and check whether an admissible input exists that satisfies (4). Analogously for InforMARL, we check whether an admissible input exists for a given $\bar{x}$ that satisfies (5).

4 Experiments and Results

Environment Setup and Probe Selection: We conduct experiments for GCBF+ and InforMARL on 2-D double-integrator dynamics. For a fair comparison of the proposed framework on GCBF+ and InforMARL, we maintain consistent training environments, with an area length $\ell = 4$, with initial agent positions, static obstacles, and goals sampled uniformly from $[0, \ell]^2$; each environment contains $N = 8$ agents and $N_{obs} = 8$ obstacles (more details in Appendix B). Following the observation in [13] that probing performance typically approaches its peak with less than 0.8% of the original training data, we train the candidate probes using 10,000 training samples, 2,500 validation samples, and 2,500 test samples. Preliminary probe comparisons (see Appendix A.1) show that linear probes are inefficient for recovering the safety property from intermediate layers. We finalize the probe architectures as MLPs with the compression schedule $[d, 16, 8, 4, 1]$ where $d = 256$ for GCBF+ and $d = 128$ for InforMARL.

Probing the CBF head of pre-trained GCBF+ indicates relatively poorer safety recoverability in terms of **Recall**, compared to **Accuracy** and **Precision** (see Figure 2). The mismatch in metrics suggests that a nontrivial subset of unsafe states lies entangled with safe and unsafe encodings in latent

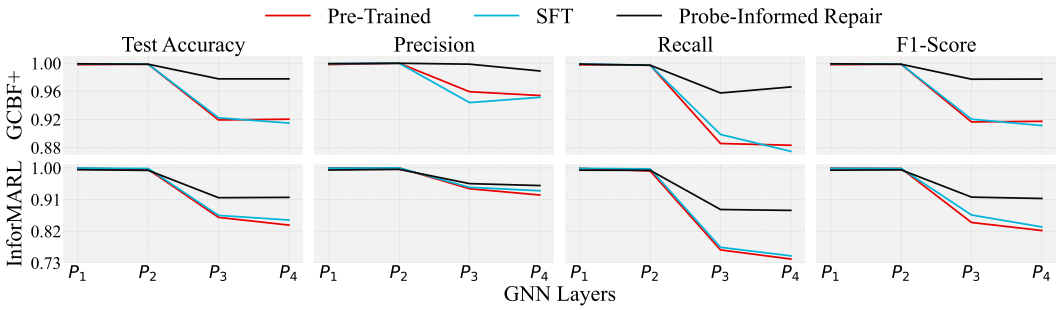


Figure 2: Probing performance across penultimate layers of all MLPs, averaged over probes trained with three different seeds. The sharp degradations across all metrics between P_2 and P_3 indicate the source of architectural error. The standard deviation is $\sim 10^{-4}$ across all metrics.

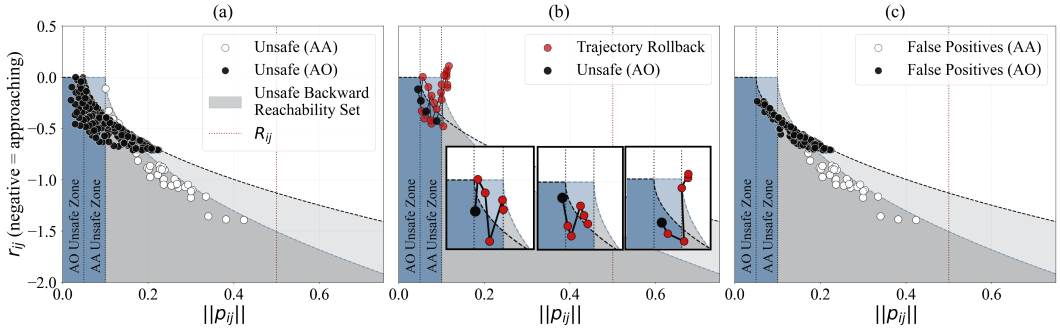


Figure 3: (a) Labeler-identified unsafe interactions by entity type, (b) arbitrary unsafe interactions rolled back by five timesteps, (c) false positive GCBF clusters by entity type.

space of GCBF+, preventing the probe from classifying them accurately. To this end, we analyze the set $\mathcal{Q}$ for GCBF+ (see Figure 3(a)), which contains realized collisions, unsafe interactions with a positive GCBF value (false-positive), and unsafe interactions with a negative GCBF value (true negatives). We evaluate the proposed repair strategies by the percentage reduction in $|\mathcal{Q}|$ and report the corresponding finish rate (which dictates system performance) as defined in [6].

Labeler-Driven Insights: Figure 3(a) shows that $\mathcal{Q}$ clusters near the AA and AO decision boundaries, with AO dominating the illustrated rollout. Two consistent patterns emerge: rollback trajectories indicate that collisions typically originate near a narrow boundary-adjacent transition region (see Figure 3(b)), and Figure 3(c) shows that GCBF+ false positives concentrate in localized regions. These results suggest underrepresentation of boundary-adjacent samples in training, thus causing misalignment between the layer encodings and the safety property.

Supervised Fine-Tuning: The labeler-driven diagnosis motivates supervised fine-tuning (SFT) [53] as a targeted repair. Since retraining with additional $\mathcal{Q}$ samples does not outperform the pre-trained model, we fine-tune the pre-trained weights using $\mathcal{Q}$ samples collected from denser environments and clipped to the original graph size. SFT uses 15% additional optimization steps and 5% of the original learning rate to mitigate catastrophic forgetting. Figure 5 shows that SFT reduces $|\mathcal{Q}|$ by 70% for GCBF+ and by 15% for InforMARL, while probe performance improves only marginally. This leaves a nontrivial residual $\mathcal{Q}$, indicating that SFT is only a partial repair.

Attention Bottleneck Analysis: Following marginal improvement through SFT, we next examine possible architectural sources of error. Figure 2 shows high recoverability of safety in Message and Aggregate MLPs, but a sharp degradation at the Update MLP, thus possibly localizing the source of error to the intervening softmax attention mechanism. Analyzing the attention weights $w_{ij} \in [0, 1]$ (see Figure 4) reveals a distance-based weight pattern, but a weaker encoding for velocity related

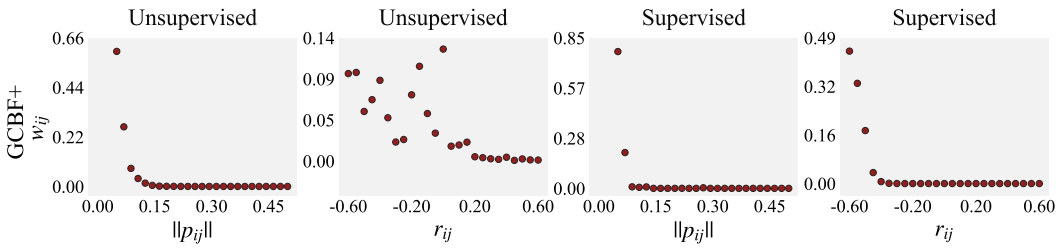


Figure 4: Unsupervised and supervised attention weights with respect to p_{ij} and r_{ij} in (1).

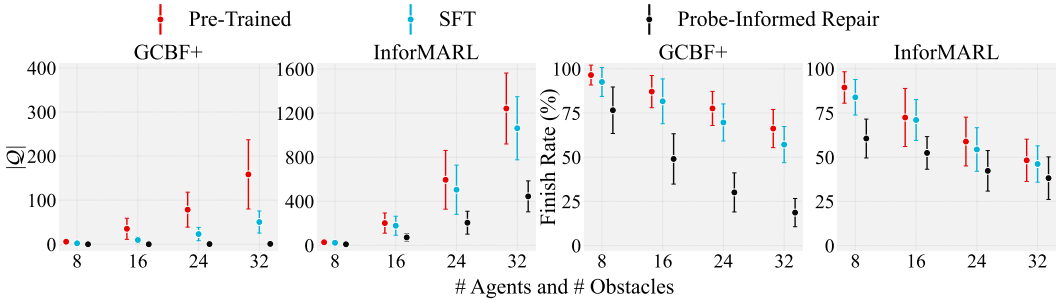


Figure 5: Mean and standard deviation of $|\mathcal{Q}|$ (see (3)) and Finish Rates of pre-trained and repaired GNN models across 32 different test environments with $l = 4$ and $N = N_{obs}$.

features. This behavior is consistent with the shift invariance of softmax [14] and the prior evidence that velocity is comparatively difficult to recover in learned controllers [31].

To determine whether the identified bottleneck arises from limited attention capacity, we first evaluate several multi-head attention (MHA) variants. Analysis of unsupervised 4-head MHA and 2-head MHA reveals head collapse, with 50% of their heads learning redundant (similar) features [54]. We then introduce orthogonal regularization (more details in Appendix C) on 2-head MHA to improve head diversity. While w_{ij} patterns for this variant exhibit improved sensitivity to velocity related features, they remain dominated by p_{ij}, and the resulting $|\mathcal{Q}|$ does not reduce. Since velocity information remains weakly represented despite increased capacity and head diversity, we next introduce supervision. Augmenting the orthogonally regularized 2-head MHA with a supervised attention head to learn r_{ij} through minimizing an added radial loss defined as $\mathcal{L}_{\text{rad}} = \sum_{j \in \mathcal{N}_i} w_{ij} r_{ij}$, subject to $\sum_{j \in \mathcal{N}_i} w_{ij} = 1$, similarly fails to yield meaningful reductions in $\mathcal{Q}$. All MHA variants underperform the pre-trained model, indicating a need for supervision. We conclude that in our setting, multi-head aggregation appears to introduce competing signals during head concatenation, which weakens the final safety representation. Finally we train a single supervised attention head with $\mathcal{L}_{rad}$, and the resulting repair reduces $|\mathcal{Q}|$ by 95% for GCBF+ and by 65% for InforMARL (see Figure 2), with improved probing performance, suggesting the lack of supervision and not limited attention capacity as the dominant source of safety degradation. We report the residual non-trivial $\mathcal{Q}$ and the uncaptured performance for InforMARL as consistent behavior since InforMARL is not designed to explicitly learn a safety structure [5]; also consistent with the findings of [17] that report that a property not directly tied with the learning objective degrades across a GNN's terminal layers.

Verification baselines: For full verification, we consider SMT-based Z3 with linear approximations for nonlinear activations, SMT-based dReal, dReal with ReLU splitting [21], and an (α, β) CROWN branch-and-bound (BaB) propagation baseline [55] (more details in Appendix D.1). **All full verification baselines** for GCBF+ and InforMARL exceed the 4 hour limit, and so, we do not report their results in the table. Since full formal verification is intractable, we compare the proposed framework against point-wise feasibility analysis, which answers analogous local safety-existence questions from a different perspective. Specifically, both determine whether safety can

Table 1: Runtime comparison (Red.%: the percentage reduction in time for probing over the baseline SMT solver) and satisfiability rates (SAT%: the percentage of queries returned as satisfiable by the baseline SMT solver) for point-wise feasibility and the proposed probing framework. Entries marked $>$ 4h exceed the 4-hour compute budget and are reported as NA.

	GCBF+			InforMARL		
Method	**Time(s)**	**Red.(%)**	**SAT%**	**Time(s)**	**Red.(%)**	**SAT%**
Point-wise Feasibility						
Z3	135.48	93.58	99.01	44.53	84.09	99.81
dReal	146.19	94.05	99.01	40.29	82.42	99.81
dReal L_∞ (2nd Order Approximation)	$>$4h	NA	NA	$>$4h	NA	NA
dReal L_∞ (1st Order Approximation)	113.92	92.30	98.24	41.04	82.74	96.63
dReal (SEEV Hierarchical Verification)	119.47	92.72	98.98	43.02	83.54	99.11
Probing Framework						
Probe End-to-End	**8.69**	–	–	**7.08**	–	–

be maintained under admissible controls: feasibility does so via the learned certificate, whereas the labeler does so via interaction features. Consequently, point-wise feasibility provides the closest verification-inspired comparison with the labeler-identified **Safe interaction rate**, defined as the percentage of safe labels generated by the labeler over the total interactions sampled by it. For point-wise feasibility, we consider Z3 and dReal, a dReal L_∞ ball variant, and a SEEV-inspired hierarchical verification baseline [41] (more details in Appendix D.2). The dReal L_∞ ball tests robustness to bounded perturbations of the agent state. The proposed framework achieves the lowest runtime overall (see Table 1), with reductions of 92.30%–94.05% for GCBF+ and 82.42%–84.09% for InforMARL. The labeler-identified **Safe interaction rates** are 98.91% for GCBF+ and 92.42% for InforMARL, sampled over the same test environment used to report Table 1.

5 Limitations

Our framework currently requires domain expertise to recognize and label the latent properties aligned with the NN objective. In robotics settings, labeling multihop information becomes harder as the NN depth increases. The repair processes we report are not exhaustive and require expert evaluation; for example, an ad hoc attempt to repair the attention framework via MHA may not be beneficial when the environment lacks sufficiently diverse factors for meaningful specializations. Comparison with formal verification is also limited by the intractability of existing tools for nonlinear dynamical GNNs. We therefore view this work as an intermediate layer between empirical sufficient checks and full formal verification, with verification of small ReLU-based probes as a natural next step. These limitations further motivate agentic artificial intelligence frameworks that automate property specification, diagnosis, and repair for application-agnostic NNs; and motivate testing the proposed framework on higher-dimensional nonlinear dynamics and different properties.

6 Conclusion

In this paper we present a probing-based diagnosis framework for neural training architectures used in design of safe MAS controls. Our framework uses intermediate GNN representations to assess whether structured safety-related information is consistently recoverable across layers, i.e., whether the safety in the representations of the GNN is aligned with its objective. The framework uses probing to localize the misalignment between the learning objective and the latent property responsible for the obective, and to guide repairs without relying on exhaustive verification. Experimental results demonstrate that probing guided repairs can achieve meaningful reductions in false modes for learned controllers, while providing a fast and tractable bridge to existing NN verification tools.

Acknowledgments

The authors of this manuscript extend their acknowledgment to Anandsingh Chauhan at Safe Robotics Group ASU for running a few experiments on SFT and to Aswin Ravikumar at Cognition and Intelligence Lab ASU for his insights on attention mechanisms and systematic ablation approaches.

References

[1] O. Shorinwa, T. Halsted, J. Yu, and M. Schwager. Distributed optimization methods for multi-robot systems: Part 2-a survey. *IEEE Robotics & Automation Magazine*, 31(3):154–169, 2024.

[2] A. Dorri, S. S. Kanhere, and R. Jurdak. Multi-agent systems: A survey. *IEEE Access*, 6: 28573–28593, 2018.

[3] C. Dawson, Z. Qin, S. Gao, and C. Fan. Safe nonlinear control using robust neural lyapunov-barrier functions. In *Conference on Robot Learning*, pages 1724–1735. PMLR, 2022.

[4] S. Zhang, K. Garg, and C. Fan. Neural graph control barrier functions guided distributed collision-avoidance multi-agent control. In *Conference on Robot Learning*, pages 2373–2392. PMLR, 2023.

[5] S. Nayak, K. Choi, W. Ding, S. Dolan, K. Gopalakrishnan, and H. Balakrishnan. Scalable multi-agent reinforcement learning through intelligent information aggregation. In *International Conference on Machine Learning*, pages 25817–25833. PMLR, 2023.

[6] S. Zhang, O. So, K. Garg, and C. Fan. Gcbf+: A neural graph control barrier function framework for distributed safe multiagent control. *IEEE Transactions on Robotics*, 41:1533–1552, 2025.

[7] J. Zhou, H. Wu, and K. Yang. Neural cooperative reach-while-avoid certificates for interconnected systems. *arXiv preprint arXiv:2601.20324*, 2026.

[8] T. Haider, K. Roscher, F. Schmoeller da Roza, and S. Günnemann. Out-of-distribution detection for reinforcement learning agents with probabilistic dynamics models. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, 2023.

[9] H. Wu, C. Barrett, M. Sharif, N. Narodytska, and G. Singh. Scalable verification of gnn-based job schedulers. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA2): 1036–1065, 2022.

[10] Y. Zhang, Z. Wei, X. Zhang, and M. Sun. Using z3 for formal modeling and verification of fnn global robustness. *arXiv preprint arXiv:2304.10558*, 2023.

[11] C. Jin. Latent causal probing: A formal perspective on probing with causal models of data. In *Conference on Language Modeling*, 2024.

[12] C. Jin and M. Rinard. Emergent representations of program semantics in language models trained on programs. In *International Conference on Machine Learning*, 2024.

[13] M. de Sousa Ribeiro, A. Leote, and J. Leite. On the performance of concept probing: The influence of the data. In *28th European Conference on Artificial Intelligence (ECAI)*, 2025.

[14] S. Gairola, M. Böhle, F. Locatello, and B. Schiele. How to probe: Simple yet effective techniques for improving post-hoc explanations. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[15] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems*, pages 41451–41530, 2023.

[16] A. Davila, J. Colan, and Y. Hasegawa. Comparison of fine-tuning strategies for transfer learning in medical image classification. *Image and Vision Computing*, 146:105012, 2024.

[17] M. S. Akhondzadeh, V. Lingam, and A. Bojchevski. Probing graph representations. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 2023.

[18] Z. Zhang, Y. Ma, Z. Cao, and H. C. Lau. Probing neural combinatorial optimization models. *Advances in Neural Information Processing Systems*, 38:131824–131862, 2026.

[19] A. Newcomb and O. Ochoa. Formal methods for safety-critical machine learning: a systematic literature review. *Frontiers in Artificial Intelligence*, 9:1749956, 2026.

[20] H. Duong, D. Xu, T. Nguyen, and M. B. Dwyer. Harnessing neuron stability to improve dnn verification. *Proceedings of the ACM on Software Engineering*, 1(FSE):859–881, 2024.

[21] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient smt solver for verifying deep neural networks. In *International Conference on Computer Aided Verification*. Springer, 2017.

[22] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2015.

[23] G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

[24] Y. Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

[25] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

[26] Y. Zheng, Y. Yuan, Y. Zhuo, Y. Li, G. Kreiman, T. Poggio, and P. Santi. Probing neural topology of large language models. *arXiv preprint arXiv:2506.01042*, 2025.

[27] S. Zhao, T. Nguyen, and A. Grover. Probing the decision boundaries of in-context learning in large language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 130408–130432, 2024.

[28] D. Zhao, X. Liu, X. Feng, H. Wang, and B. Qin. Probing and boosting large language models capabilities via attention heads. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 28518–28532, 2025.

[29] A. Sclocchi, A. Favero, N. Levi, and M. Wyart. Probing the latent hierarchical structure of data via diffusion models. In *International Conference on Learning Representations*, volume 2025, pages 72703–72732, 2025.

[30] M. Zhao, X. Huang, S. Zhang, A. Liu, Z. Lyu, Y. Wang, L. Cui, and L. Bai. Graphprobe: Knowledge probing for graph representation learning. *Pattern Recognition*, page 112518, 2025.

[31] X. Chen, Z. Zhu, and A. Perrault. Understanding learned representations and action collapse in visual reinforcement learning. In *Reinforcement Learning Conference*, 2025.

[32] T.-H. Wang, W. Xiao, T. Seyde, R. Hasani, and D. Rus. Measuring interpretability of neural policies of robots with disentangled representation. In *Conference on Robot Learning*, pages 602–641. PMLR, 2023.

[33] M. Taghian, S. Miwa, Y. Mitsuka, J. Günther, S. Golestan, and O. Zaiane. Explainability of deep reinforcement learning algorithms in robotic domains by using layer-wise relevance propagation. *Engineering Applications of Artificial Intelligence*, 137:109131, 2024.

[34] Y. Septon, T. Huber, E. André, and O. Amir. Integrating policy summaries with reward decomposition for explaining reinforcement learning agents. In *International Conference on Practical Applications of Agents and Multi-Agent Systems*, pages 320–332. Springer, 2023.

[35] X. Sun and Y. Shoukry. Neurosymbolic motion and task planning for linear temporal logic tasks. *IEEE Transactions on Robotics*, 40:2749–2768, 2024.

[36] M. Yuasa, R. S. Sreenivas, and H. T. Tran. Neuro-symbolic generation of explanations for robot policies with weighted signal temporal logic. *IEEE Robotics and Automation Letters*, 2026.

[37] L. Fernández-Becerra, D. Sobrín-Hidalgo, M. A. González-Santamarta, Á. Manuel Guerrero-Higueras, F. J. Rodríguez Lera, and V. M. Olivera. Improving accountability and explainability in robots through encryption, large language models, and visual language models. *Logic Journal of the IGPL*, 34(1):jzaf016, 2026.

[38] M. T. Akbulut, V. Satheesh, A. Jaafar, A. Ahmetoglu, S. Parr, A. Ganeshan, S. Vats, and G. Konidaris. Creative robot tool use by counterfactual reasoning. *arXiv preprint arXiv:2605.05411*, 2026.

[39] H. Zhao, X. Zeng, T. Chen, Z. Liu, and J. Woodcock. Learning safe neural network controllers with barrier certificates. *Formal Aspects of Computing*, 33(3):437–455, 2021.

[40] U. Mandal, G. Amir, H. Wu, I. Daukantas, F. L. Newell, U. J. Ravaioli, B. Meng, M. Durling, M. Ganai, T. Shim, et al. Formally verifying deep reinforcement learning controllers with lyapunov barrier certificates. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*, pages 95–106. IEEE, 2024.

[41] H. Zhang, Z. Qin, S. Gao, and A. Clark. Seev: Synthesis with efficient exact verification for relu neural barrier functions. *Advances in Neural Information Processing Systems*, 37: 101367–101392, 2024.

[42] T. Ladner, M. Eichelbeck, and M. Althoff. Formal verification of graph convolutional networks with uncertain node features and uncertain graph structure. *Transactions on Machine Learning Research*, 2025.

[43] M. Liu, C.-H. Lu, and M. Kwiatkowska. Exact verification of graph neural networks with incremental constraint solving. In *International Symposium on Formal Methods*, pages 641–662. Springer, 2026.

[44] A. Chernobrovkin, M. Sälzer, F. Schwarzentruber, and N. Troquard. Verifying graph neural networks with readout is intractable. *arXiv preprint arXiv:2510.08045*, 2025.

[45] M. Yu, C. Yu, M.-M. Naddaf-Sh, D. Upadhyay, S. Gao, and C. Fan. Efficient motion planning for manipulators with control barrier function-induced neural controller. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14348–14355. IEEE, 2024.

[46] W. Xiao, C. G. Cassandras, and C. A. Belta. Learning feasibility constraints for control barrier functions. In *2023 European Control Conference (ECC)*, pages 1–6. IEEE, 2023.

[47] J. Zeng, B. Zhang, Z. Li, and K. Sreenath. Safety-critical control using optimal-decay control barrier function with guaranteed point-wise feasibility. In *American Control Conference (ACC)*, pages 3856–3863. IEEE, 2021.

[48] L. Wang, A. D. Ames, and M. Egerstedt. Safety barrier certificates for collisions-free multirobot systems. *IEEE Transactions on Robotics*, 33(3):661–674, 2017.

[49] O. Rainio, J. Teuho, and R. Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14(1):6086, 2024.

[50] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2016.

[51] Y. Chen, M. Jankovic, M. Santillo, and A. D. Ames. Backup control barrier functions: Formulation and comparative study. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 6835–6841. IEEE, 2021.

[52] A. Clark. Verification and synthesis of control barrier functions. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 6105–6112. IEEE, 2021.

[53] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[54] Y. Chen, L. Zhang, J. Shang, Z. Zhang, T. Liu, S. Wang, and Y. Sun. Dha: Learning decoupled-head attention from transformer checkpoints via adaptive heads fusion. *Advances in Neural Information Processing Systems*, 37:45879–45913, 2024.

[55] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter. Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. *Advances in Neural Information Processing Systems*, 34:29909–29921, 2021.

Appendix

A Probing Details

A.1 Probing Architectures and Performances.

Table 2 provides the complete hyperparameter configuration used to train all candidate probes. We keep this configuration fixed across probes to compare their expressive capacity under a common training protocol, rather than tuning each probe independently, which could introduce additional inductive bias or overfitting. We do not normalize the input data because all samples originate from a single source and consist of GNN layer embeddings that are already comparable in scale. In our setting, normalization obscures the safety signal and leads to perfect (100%) performance across evaluation metrics, which is indicative of undesirable overfitting. However, normalization remains a useful default in general and may be appropriate in other settings. We use a standard 70/15/15 split for training, validation, and testing. For training and evaluation, we maintain a balanced 50/50 distribution of safe and unsafe interactions, since unsafe samples can be generated in sufficient quantity by varying the environment seed, number of agents, and number of obstacles, thereby preserving the integrity of the data distribution.

Table 2: Training hyperparameters used for all all candidate probes.

Hyperparameter	**Value**
Learning Rate	10^{-3}
Batch Size	32
Optimizer	Adam
Weight Decay	None
Epochs	500
Dropout	None
Normalization	None
Dense Layer Activation Function	Leaky ReLU ($\alpha = 0.01$)
Output Activation Function	Sigmoid

Table 3 and Table 4 report the performance of each candidate probe, averaged over three training runs with different random seeds for GCBF+ and InforMARL, respectively. Unless otherwise stated, we report **Accuracy**, **Precision**, and **Recall** on the test set. **Recall** is the primary criterion for probe selection in our setting, as discussed in Section 3, since failing to identify unsafe states as unsafe is more consequential than conservatively labeling safe states as unsafe.

We observe that MLPs with increasing layer depth outperform linear probes across both case studies in all metrics. Adding more depth in the dense layers helps improve performance but increasing the layer size for more expressivity yields only marginal improvements (see Table 4), if any at all (see Table 3), hence we do not select the higher complexity probe in case of marginal improvement to avoid overfitting. In exploring alternative MLP probe designs, we restrict hidden-layer widths to be no larger than the input dimensionality, to limit unnecessary increases in model capacity and reduce the risk of overfitting. Across all the reported achitectures, **Validation Accuracy** (Val. Acc.) does not diverge from the **Test Accuracy** (Test Acc.) and indicates no obvious signs of overfitting. Exploring other MLP probe designs, we choose not to evaluate probes with hidden layer dimensions more than the input dimensions, for the same reason of avoiding overfitting.

A.2 Effect of Training Data Size on the Selected Probes

Table 5 and Table 6 report the performance of the selected probes for GCBF+ and InforMARL, respectively, under different sample sizes used for probe training and evaluation, where sample size refers to the aggregated set of the training, validation, and test splits. We observe that, for linear probes, 10,000 samples are sufficient to converge to a stable performance. However, for the MLP probes, performance with 15,000 samples shows performance gains over 10,000 samples, indicating

Table 3: Probe evaluation for GCBF+ across different probe categories and compression schedules.

Dimsensions	Val. Acc.	Test Acc.	Precision	Recall	F1-Score
Linear Probe					
256-1	0.8811±0.0001	0.8755±0.0005	0.9198±0.0009	0.8236±0.0010	0.8687±0.0006
MLP Probe [Input dimensions - Hidden dimensions - Output dimensions]					
256-10-5-1	0.9131±0.0006	0.9034±0.0024	0.9336±0.0091	0.8689±0.0050	0.8998±0.0019
256-16-4-1	0.9061±0.0088	0.8956±0.0083	0.9456±0.0018	0.8394±0.0195	0.8893±0.0100
256-16-8-4-1	0.9242±0.0050	0.9204±0.0052	0.9541±0.0096	0.8833±0.0012	0.9174±0.0048
256-16-16-4-1	0.9224±0.0061	0.9158±0.0037	0.9591±0.0166	0.8689±0.0170	0.9116±0.0037
256-32-16-4-1	0.9217±0.0061	0.9111±0.0061	0.9515±0.0067	0.8668±0.0191	0.9070±0.0076

Table 4: Probe evaluation for InforMARL across different probe categories and compression schedules.

Dimensions	Val. Acc.	Test Acc.	Precision	Recall	F1-Score
Linear Probe					
128-1	0.7904 ±0.0003	0.7980±0.0022	0.8913±0.0012	0.6898±0.0005	0.7777±0.0090
MLP Probe [Input dimensions - Hidden dimensions - Output dimensions]					
128-10-5-1	0.8224±0.0027	0.8162±0.0012	0.8978±0.0071	0.7264±0.0042	0.8030±0.0037
128-16-4-1	0.8154±0.0043	0.8084±0.0064	0.9098±0.0016	0.7069±0.0127	0.7956±0.0088
128-16-8-4-1	0.8335±0.0047	0.8374±0.0037	0.9230±0.0041	0.7408±0.0178	0.8219±0.0029
128-16-16-4-1	0.8317±0.0052	0.8286±0.0042	0.9306±0.0131	0.7364±0.0098	0.8221±0.0076
128-32-16-4-1	0.8310±0.0027	0.8322±0.0024	0.9250±0.0251	0.7433±0.0316	0.8242±0.0106

that 10,000 samples may lead to underfitting. In contrast, increasing the training size from 15,000 to 20,000 yields no meaningful performance gains. We also observe consistent validation and test accuracies, which indicates no obvious signs of overfitting; however, this observation alone does not eliminate the possibility of overfitting. Thus, to avoid potential overfitting, we select a sample size of 15,000, especially when gains with further increase in the sample size are nonexistent or marginal.

Table 5: MLP (256-16-8-4-1) performance for GCBF+ on different sample sizes.

Sample Size	Test Accuracy	Test Precision	Test Recall	Test F1-Score
10,000	0.9127 ± 0.0042	0.9210 ± 0.0037	0.8306 ± 0.0028	0.9099 ± 0.0043
15,000	0.9204 ± 0.0052	0.9541 ± 0.0096	0.8833 ± 0.0012	0.9174 ± 0.0048
20,000	0.9246 ± 0.0020	0.9393 ± 0.0129	0.8893 ± 0.0239	0.9138 ± 0.0064

A.3 Data for Figure 2

For the sake of clarity, we provide the data for Figure 2 in Tables 7 and 8 for GCFB+ and InforMARL, respectively.

Table 6: MLP (128-16-8-4-1) performance for InforMARL on different sample sizes.

Sample Size	Test Accuracy	Test Precision	Test Recall	Test F1-Score
10,000	0.7839 ± 0.0031	0.9071 ± 0.0027	0.7153 ± 0.0095	0.7998 ± 0.0021
15,000	0.8374 ± 0.0037	0.9230 ± 0.0041	0.7408 ± 0.0178	0.8219 ± 0.0029
20,000	0.8351 ± 0.0163	0.9262 ± 0.0132	0.7458 ± 0.0347	0.8251 ± 0.0245

Table 7: GCBF+ probing data for Figure 2

GNN Layer	Test Accuracy	Test Precision	Test Recall	Test F1-Score
Pre-Trained				
Message	0.9980	0.9982	0.9978	0.9980
Aggregate	0.9986	1.0000	0.9972	0.9986
Update	0.9193	0.9595	0.8858	0.9165
CBF head	0.9204	0.9541	0.8833	0.9174
SFT				
Message	0.9992	0.9994	0.9988	0.9992
Aggregate	0.9986	1.0000	0.9972	0.9986
Update	0.9223	0.9440	0.8987	0.9204
CBF head	0.9149	0.9515	0.8746	0.9113
Probe-Informed Repair				
Message	0.9992	0.9994	0.9988	0.9992
Aggregate	0.9986	1.0000	0.9972	0.9986
Update	0.9778	0.9987	0.9577	0.9773
CBF head	0.9778	0.9890	0.9663	0.9775

Table 8: InforMARL probing data for Figure 2

GNN Layer	Test Accuracy	Test Precision	Test Recall	Test F1-Score
Pre-Trained				
Message	0.9994	1.0000	0.9988	0.9993
Aggregate	0.9957	1.0000	0.9914	0.9985
Update	0.8593	0.9408	0.7667	0.8448
Critic head	0.8374	0.9230	0.7408	0.8219
SFT				
Message	0.9997	0.9994	0.9988	0.9990
Aggregate	0.9984	1.0000	0.9972	0.9985
Update	0.8649	0.9449	0.7741	0.8661
Critic head	0.8519	0.9351	0.7498	0.8322
Probing-Informed Repair				
Message	0.9950	0.9941	0.9937	0.9938
Aggregate	0.9931	0.9959	0.9932	0.9945
Update	0.9153	0.9554	0.8817	0.9170
Critic head	0.9163	0.9499	0.8792	0.9131

B Training and Testing Environments

B.1 DoubleIntegrator Environment

In this environment, the state of the agent i is given by $x_i = [p_i^x, p_i^y, v_i^x, v_i^y]^\top$, where $[p_i^x, p_i^y]^\top$ is the position and $[v_i^x, v_i^y]^\top$ is the velocity. We use $e_{ij} = x_i - x_j$ as the edge information between agent i and neighbor j .The control input of the agent is given by $u_i = [a_i^x, a_i^y]^\top$, i.e. the acceleration. The dynamics function is given by:

$$\dot{x}_i = [v_i^x, v_i^y, a_i^x, a_i^y]^\top, \tag{6}$$

with the simulation timestep $\delta t = 0.03$s. The control input is bounded as $\|u\| \leq u_{\max}$ as shown in Table 9.

B.2 Implementation Details

In our implementation of GCBF+ and InforMARL, the entity-specific safety thresholds, as mentioned in Section 3 are defined as follows:

- **Safety (Obstacles):** $\|p_i(t) - p_j(t)\| > r_i,\ \forall j = 1, \ldots, n_{\text{rays}}, \forall t \geq 0$, i.e., the agents do not collide with obstacles.
- **Safety (Other Agents):** $\|p_i(t) - p_j(t)\| > 2r_i, \forall t \geq 0$ and $j \neq i$, i.e., the agents do not collide with each other.

Here, $r_i = r \in \mathbb{R}_{>0}$ denotes the radius of agent i, and $n_{\text{rays}} \in \mathbb{N}_{>0}$ denotes the number of sensor rays (LiDAR in our setting) radially emitted by agent i to capture relative state features of entity j within R_i. Our implementation consists of only homogeneous agents and static obstacles, however, the two frameworks are directly extendable to heterogeneous agents and dynamic obstacles. Various parameters used in the implementation are reported in Table 9

Table 9: Training and testing configurations used for GCBF+ and InforMARL.

Configuration	GCBF+	InforMARL
Pre-Trained		
Number of Agents (N)	8	8
Number of Obstacles (N_{obs})	8	8
Agent Sensing Radius (R_i)	0.5	0.5
Agent Radius (r)	0.05	0.05
Area length (ℓ)	4	4
Agent Control Bound ($u_{i\,\max}$)	1	1
Batch Size	256	8192
Number of Training Environments	16	32
Number of Testing Environments	32	32
Number of LiDAR rays	32	32
Training Steps	1000	15000
Actor Epochs per Training Step	8	10
GCBF/Critic Epochs per Step	8	3
Buffer Size	512	512
Horizon	32	32
Max Timestep (T)	256	256
Actor Learning Rate	1e-5	3e-4
GCBF/Critic Learning Rate	1e-5	3e-5
Number of GNN Layers	1	1
SFT		
Actor Learning Rate	1e-5	0.05
GCBF/Critic Learning Rate	5e-7	15e-7
Additional Training Steps	150	2000
Probe-Informed Repair		
Radial Loss Learning Rate	1e-1	1.5e-1

C Probe-Informed Repair

In this section, we present systematic ablations (see Figure 6) conducted to repair a possible architectural source of error identified through probing (see Figure 2) for GCBF+. The probing results indicate that the unsupervised single-head attention mechanism learns a predominantly distance-based function (see Figure 4), with little sensitivity to the r_{ij}, even though both are important for determining safety; for example: a neighbor j_2 farther from i than j_1 with $r_{ij}^2 < r_{ij}^1$ ($r_{ij}^2, r_{ij}^1 < 0$) may pose a bigger safety threat to i than j_1, which the pre-trained attention head may completely disregard and focus more on j_1.

As a first step to determine if the existing attention's representational capacity is limited, we consider an unsupervised 4-head MHA variant, where the outputs of all heads are combined through linear concatenation. This setting reveals clear head collapse: two pairs of heads learn very similar weighting patterns across distance and velocity features, with head 2 nearly identical to head 1 and head 4 closely matching head 3 (see Figure 7). These results suggest that the environment does not provide sufficient interaction diversity for all heads to specialize meaningfully. To this end, we next consider a 2-head unsupervised attention model with linear concatenation, which still does not outperform the pre-trained model and does not learn any sensitivity to velocity features (see Figure 8).

Following the failure of unsupervised attention, we next encourage greater representational diversity across heads, we then introduce an orthogonal regularization term, $\mathcal{L}_{\text{orth}}$, in the 2-head MHA setting. We define the column-normalized head embeddings as:

$$\tilde{W}_{\bar{k}}(:, \bar{q}) = \frac{W_{\bar{k}}(:, \bar{q})}{\|W_{\bar{k}}(:, \bar{q})\|_2 + \varepsilon}, \qquad \bar{k} \in \{0, 1\}, \tag{7}$$

and the orthogonal regularizer as

$$\mathcal{L}_{\text{orth}} = \frac{\lambda}{d^2} \left\| \tilde{W}_0^\top \tilde{W}_1 \right\|_F^2. \tag{8}$$

Here, W_0 and W_1 denote the embedding matrices of the two heads, $\tilde{W}_0$ and $\tilde{W}_1$ denote their column-normalized forms, and d is the head dimension, and $||A||_F$ represents the Frobenius norm of matrix A. Minimizing this term discourages duplicate feature extraction across heads and encourages the heads to specialize in distinct components of the interaction signal. While this variant shows some increased sensitivity to velocity-related features, the resulting attention patterns remain diffuse (see Figure 9) and do not yield meaningful improvements over the pre-trained model. We next combine the orthogonally regularized 2-head MHA with a supervised radial loss, $\mathcal{L}_{\text{rad}}$, applied to one of the heads. However, this setting also fails to outperform the pre-trained model, likely because the two heads still compete in forming the final attention signal (see Figure 10).

Finally, we study a single-head attention model supervised directly with $\mathcal{L}_{\text{rad}}$, and report the results in Figure 11. This variant performs best among the attention ablations for two reasons. First, it removes competition between heads when constructing the final attention weights. Second, supervising a single head to recover r_{ij} is sufficient in this setting, since distance and the radial component of relative velocity are strongly coupled and do not require multiple heads to specialize separately. As a result, the single-head supervised model produces cleaner attention signals and the strongest overall repair behavior.

Figure 12 shows that the pre-trained attention in InforMARL exhibits diffuse sensitivity to r_{ij} and little sensitivity to distance features. At the reported learning rate in Table 9, the supervised head recovers the distance feature cleanly, although the learned response to r_{ij} remains limited because $\mathcal{L}_{\text{rad}}$ competes with reward maximization. Increasing the coefficient of $\mathcal{L}_{\text{rad}}$ beyond the reported value further strengthens this competition, causing the critic to fail to converge for thousands of steps and makes the training unstable.

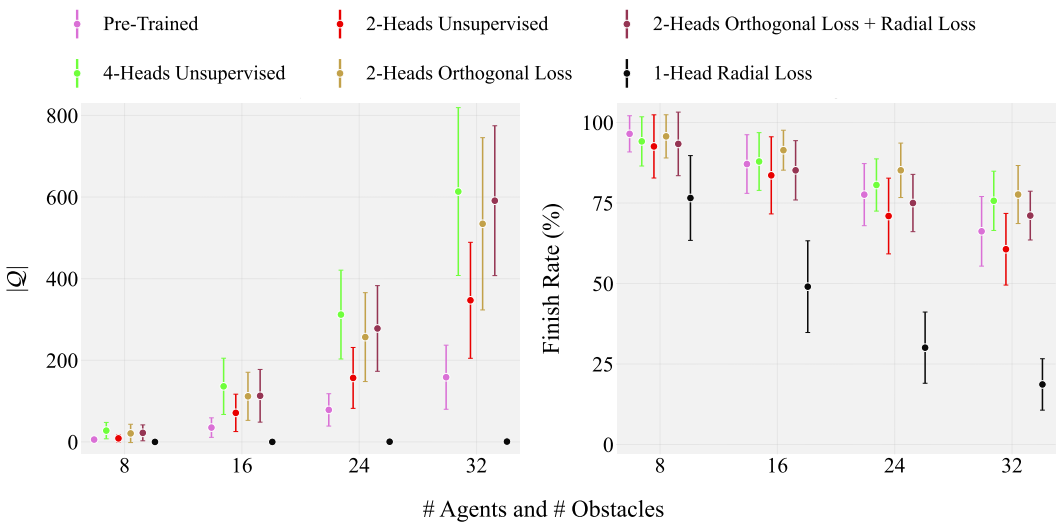


Figure 6: Mean and std. of $|\mathcal{Q}|$ (defined in (3)) and Finish Rates of 6 different classes of attention models across 32 different test environments with $l = 4$ and $N = N_{obs}$. 2-head unsupervised MHA performs closest to the pre-trained single head attention among all variants of MHA, while supervised single head attention with $\mathcal{L}_{rad}$ outperforms all the variants of attention.

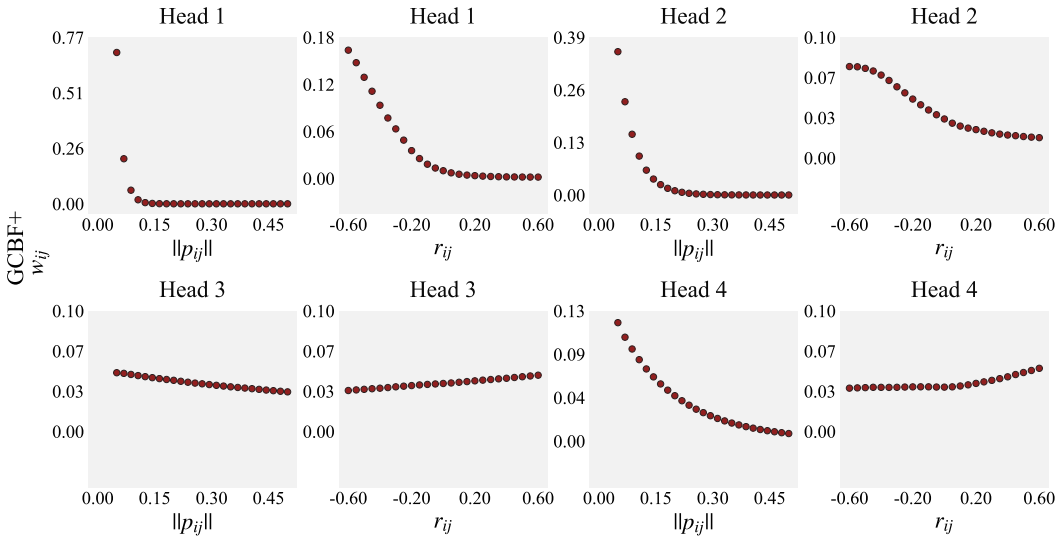


Figure 7: Attention weights with respect to p_{ij} and r_{ij} for individual heads, for unsupervised 4-head MHA. Heads 1 and 2 show a similarly learned pattern, while heads 3 and 4 show a similar pattern.

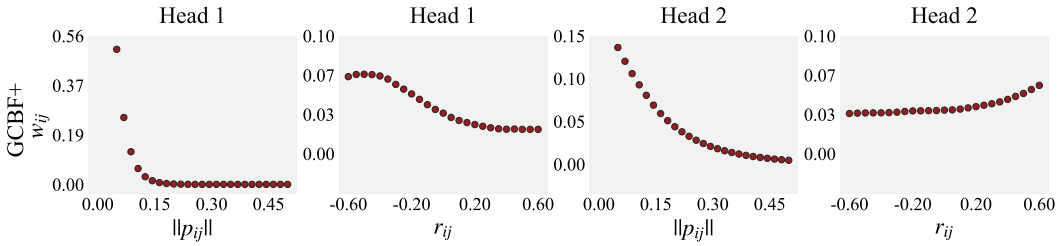


Figure 8: Attention weights with respect to p_{ij} and r_{ij} for individual heads, for unsupervised 2-head MHA. Both heads learn a correct distance based weight pattern but neither learns the correct sensitivity to r_{ij}.

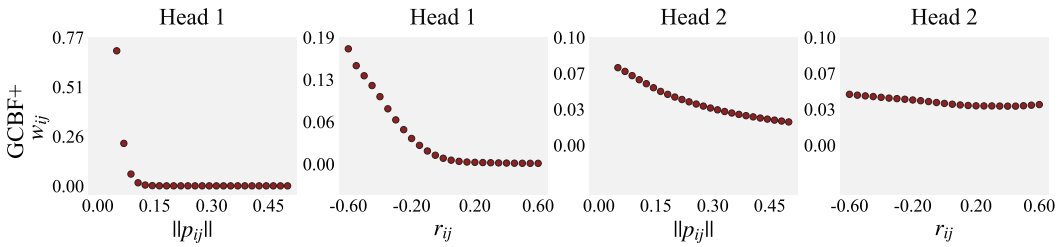


Figure 9: Attention weights with respect to p_{ij} and r_{ij} for individual heads, for 2-head MHA with orthogonal regularization between heads.

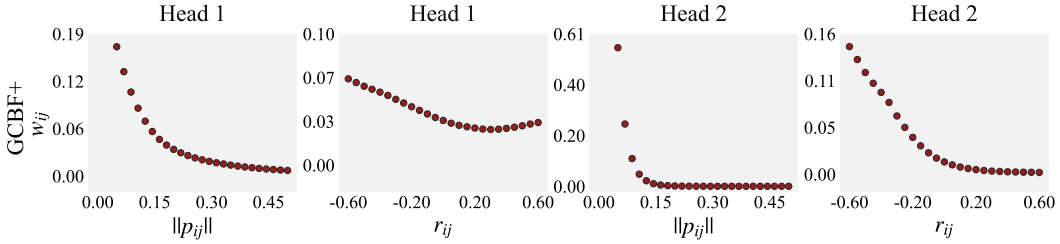


Figure 10: Attention weights with respect to p_{ij} and r_{ij} for individual heads, for 2-head MHA with orthogonal regularization between heads and L_{rad} applied to Head 2.

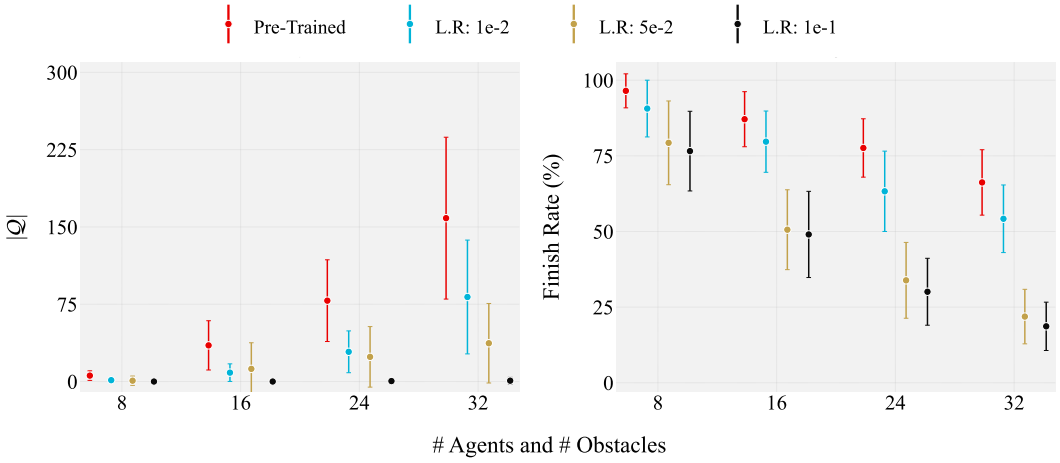


Figure 11: Mean and std. of $|\mathcal{Q}|$ (defined in (3)) and Finish Rates of pre-trained and radial attention GNN models across 32 different test environments with $l = 4$ and $N = N_{obs}$. As the learning rate increases, the agents converge to better safety but at the increased cost of conservative behavior.

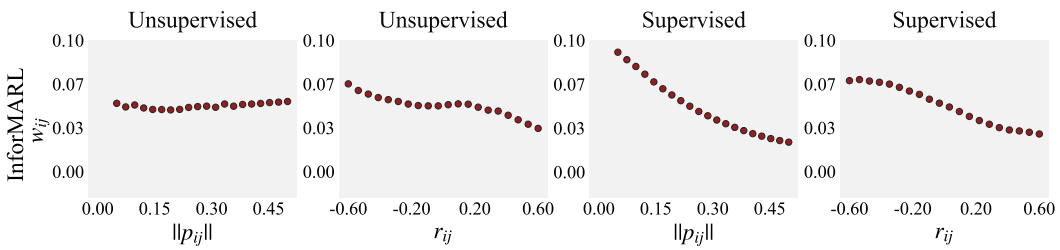


Figure 12: Unsupervised and supervised attention weights with respect to p_{ij} and r_{ij} in (1).

D Verification Implementation Details

D.1 Full Formal Verification

We compare the proposed framework against full formal verification, which answers whether the conditions in (4) and (5) hold over the entire verification domains $\mathcal{C}$ and $\mathcal{D}$ for GCBF+ and InforMARL respectively, for each timestep $t \in \mathcal{T}$ and each agent $i \in \mathcal{V}$, rather than only at sampled rollout states x_i. Specifically, for each agent $i \in \mathcal{V}$, the verifier searches over the state-domain bounds (the verification domain) used in the solver, and checks whether the verifier condition can be violated anywhere in that domain.

We first consider a Z3 SMT baseline, following the baseline setup of [41]. Since Z3 cannot directly encode the nonlinearities in the learned networks, it not directly applicable to the GNNs in GCBF+ and InforMARL. We approximate $\tanh$ and softmax activations using linear upper and lower bounds given by a chord upper bound and a tangent lower bound on the relevant interval. The resulting constraints are then given to Z3 as a satisfiability query over the verifier domain.

We next consider a dReal SMT baseline, following the baseline setup of [41]. dReal handles the nonlinearities via δ-satisfiability, with $\delta = 10^{-3}$. This baseline does not require explicit piecewise linear approximations, but it remains computationally expensive because it must reason over the full network and the verifier domain.

We additionally consider a dReal baseline with ReLU splitting and interval propagation [21]. Here, interval arithmetic is used to bound each ReLU pre-activation, and each neuron is classified as active, inactive, or unstable, where the three cases are defined as:

- **Active** if $\mathrm{ReLU}(z) = z, \forall z \in [z^\ell, z^u]$, equivalently $z^\ell \geq 0$,
- **Inactive** if $\mathrm{ReLU}(z) = 0, \forall z \in [z^\ell, z^u]$, equivalently $z^u \leq 0$.
- **Unstable** if neither of the above holds, equivalently $z^\ell < 0 < z^u$,

The verifier then branches only on unstable neurons, producing a Reluplex-style case split over the nonlinear subproblems. The active and inactive neurons are resolved (i.e. deterministic behavior) directly from interval bounds, while unstable neurons induce case splits; the solver recursively explores these subproblems until the verification condition is decided.

Finally, we consider a branch-and-bound baseline with CROWN-style bound propagation [55]. The verifier does not operate on GNN embeddings directly, but instead verifies the learned GNN output as a function of the local pairwise edge feature $e_{ij}(t)$. The verifier propagates interval bounds through the Message, Attention, Update, and framework-specific head layers, classifies ReLU pre-activations as active, inactive, or unstable, and then backpropagates linear relaxations to obtain lower and upper certificates on the scalar network output. The CROWN relaxation yields coefficients (A_ℓ, b_ℓ) and (A_u, b_u) such that $A_\ell e_{ij} + b_\ell \leq f_\theta(e_{ij}) \leq A_u e_{ij} + b_u$, where f_θ denotes the scalar pre-$\tanh$ output of the GNN. If this certificate is not sufficient to decide the verification condition on the box, the widest coordinate interval is split into two sub-boxes and the procedure is repeated recursively. The unresolved boxes continue to split until either the depth limit is reached or the 4-hour compute budget expires, and we report these timeouts separately. Without a maximum branch depth, the 4-hour compute budget gets exhausted and with a maximum branch depth of 30, the depth limit is reached and the verification process terminates.

All four full-verification baselines are evaluated on the same rollout and exceed the 4-hour compute budget and are therefore characterized as timeouts and not reported in Table 1.

D.2 Point-wise Feasibility

The proposed framework against point-wise feasibility asks whether, at each sampled rollout state, there exists an admissible control input satisfying the framework-specific conditions in (4) and (5).

Unlike full formal verification, this is a local safety existence check over visited states and does not certify correctness over the full safe set.

For each timestep $t \in \mathcal{T}$ and each agent $i \in \mathcal{V}$, we evaluate the quantity appearing in the point-wise condition at the sampled rollout state, namely $h_i(\bar{x}_{\mathcal{N}_i})$ for GCBF+ and $\dot{V}_i(\bar{x})$ for InforMARL. We then use JAX automatic differentiation to compute its gradient with respect to the x_i and combine it with the control-affine dynamics to obtain the corresponding Lie-derivative terms in (4) and (5). Since the feasibility check is posed per agent under the decentralized 1-hop formulation, only the derivative with respect to agent i's own state x_i is used when constructing the solver constraints.

We use Z3 and dReal as point-wise feasibility baselines. Z3 checks exact satisfiability of the control existence problem, whereas dReal provides a δ-satisfiability check with $\delta = 10^{-3}$, which we use to account for numerical imprecision near the feasibility boundary induced by the underlying GNNs.

We additionally consider a dReal L_∞-ball baseline to test local robustness under bounded state perturbations. Here, L_∞ denotes coordinate-wise perturbations satisfying $\|\delta x_i\|_\infty \leq \epsilon$, so that each component of the agent state may vary independently within $[-\epsilon, \epsilon]$; we use $\epsilon = 0.1$ for robustness checks.

We also evaluate a second-order dReal L_∞-ball variant, obtained by retaining the Hessian terms in the local Taylor expansion of the quantity appearing in the point-wise condition around x_i. The resulting nonlinear satisfiability check is more expensive and exceeds the 4-hour compute budget in our experiments.

We further include a SEEV-inspired hierarchical baseline [41]. For each agent i and x_i, the first stage checks whether the control term in (4) and (5) can be made sufficiently large using $u_{i\,\max}$ so that the point-wise condition is instantly certified at x_i. This provides a conservative sufficient check: if it fails, then no admissible control can satisfy the point-wise condition at x_i. If the first stage check passes, we construct a finite set of candidate controls by discretizing each action dimension into a small grid, propagate the dynamics one step from x_i under each candidate control, and reevaluate the same condition at the corresponding successor states. The state x_i is certified by this baseline only if the base-state check passes and no violation is found among the one-step screened successors; otherwise, it is left uncertified.

All baselines are evaluated on the same rollout and under the same per-agent protocol, differing only in whether they test exact feasibility, δ-feasibility, bounded-perturbation robustness, or a hierarchical neighborhood certificate.

E Hardware Used

All experiments are conducted on a Dell Pro Max laptop equipped with an NVIDIA RTX Pro 5000 Blackwell GPU with 24 GB of memory. The system further includes an Intel Core Ultra 9 285H vPro Enterprise CPU (24 cores, up to 5.40 GHz), and 30 GB of RAM. Trainings for GCBF+ and InforMARL are performed on the GPU. In contrast, the testing for trained checkpoints, verifications, probe training, and probe testing are performed on the CPU to maintain reproducability and a fair comparison across the results reported in Table 1. All experiments are implemented usig JAX/jaxlib, specifically jaxlib version 0.6.2, with CUDA 12 support wherever applicable.